Skip to main content

ILLINOIS

Search in course

Search

- English
-
- Menu pop-up Collapsed

B

Hide menu

- **Week 7 Information**
- **Scalable Data Storage**
- **Caching and Key-Value Stores**
- **Message Queues / Publish - Subscribe Queues**
- **Apache Kafka**
- **Week 7 Graded Activities**
  -

Graded Assignment: Week 7 Quiz

. Duration: 1 hour1h

  -

Reading: ReadingHBase Tutorial

. Duration: 10 minutes10 min

○

Programming Assignment: Machine Problem 7 V2: HBase

. Duration: 6 hours6h

1. CS 498: Cloud Computing Applications
2. Week 7
3. Machine Problem 7 V2: HBase

PreviousNext

Programming Assignment: Machine Problem 7 V2: HBase

Graded • 8/8 points

Deadline

Pass this assignment by Apr 6, 11:59 PM CDT

InstructionsMy submissionsDiscussions

1. Overview

Welcome to HBase Machine Problem. This MP has both Java and python templates. Please use whichever you prefer.

We recommend looking at the **Tutorial: Introduction to HBase** before beginning this assignment. Note that the tutorial refers to HDP 2.3 distro and is from 2015. Since then, we have discontinued using the HDP environment in our course, but reading the tutorial and becoming familiar with the concepts is still beneficial.

2. General Requirements

Please note that our grader runs on a docker container **NOT connected to the internet**. Therefore, **no additional libraries are allowed** for this assignment. Also, you will **NOT be allowed to create any file or folder outside the current folder** (that is, you can only create files and folders in the folder that your solutions are in).

3. Set Up HBase

Step 1: Download the files for this MP, change the current folder, build, and run the docker image, run:

1

2

3

4

git clone https://github.com/UIUC-CS498-Cloud/MP7_HBase_Template.git

cd MP7_HBase_Template/Docker

docker build -t mp7 .

docker run -it mp7 bin/bash

For Windows user, before building MP7 docker image, please first do end of line (EOL) conversion for hbase-env.sh under MP7_HBase_Template/Docker/ from CRLF to LF before building your docker. For notepad++ users: go to edit -> EOL conversion -> change from CRLF to LF. For VScode users: you can change CRLF to LF by clicking on CRLF present on lower right side in the status bar.

Step 2: start Zookeeper server start HBase, run:

1

2

zkServer.sh start

start-hbase.sh

Java submission

If you choose to do this assignment in Python, skip this part and go to the **"Python submission" section below.

1. Requirements

This assignment will be graded based on **JDK 11.** The version of HBase in the Docker container is **2.4.16.** You may find helpful information on how to solve the following exercise by referring to the official documentation for Apache HBase, available at

https://hbase.apache.org/apidocs/overview-summary.html.

2. Procedures

Step 1: Now we have to copy the template from our local folder to our container :

1

2

3

4

5

6

7

8

#open a new command prompt and get the container ID by running:

docker ps -a

#copy a local file into container

docker cp [OPTIONS] SRC_PATH CONTAINER:DEST_PATH

#docker cp C:\Users\lenovo\Desktop\MP7_HBase_Template\JavaTemplate 7cdd497f0560:/JavaTemplate

#In the docker terminal change the current folder and run your files:

cd JavaTemplate

Step 2: Set the environment variable CLASSPATH:

1

2

export CLASSPATH=$(hbase classpath)

export CLASSPATH="/JavaTemplate:${CLASSPATH}"

Step 3: Finish the exercises by editing the provided template files. All you need to do is complete the parts marked with **TODO**. Please note that you are **NOT allowed to import any additional libraries.**

- Each exercise has a Java code template. All you must do is edit this file.

- Each exercise should be implemented in one file only. Multiple file implementation is not allowed.
- Compile and run the code on the provided Docker image.
- You should run the parts in the order described below, or you may not get the correct results. For example, you should create the tables first and then list them.
- Remember that the output is case-sensitive. Furthermore, your application should not remove the table and its values at the end of execution.

Exercise A: Create HBase Tables

In this exercise, you will create two tables, as shown below. We have provided a template for this exercise in the following file: **TablePartA.java**.

Table 1: a table named "powers" in HBase with the three column families below. For simplicity, treat all data as **String** (i.e., don't use any numerical data type). Set max versions count per column family to 3.

row key	personal	professional	custom

Table 2: a table named "food" in HBase with two column families below. For simplicity, treat all data as **String** (i.e., don't use any numerical data type).

row key	nutrition	taste

We use the following commands to run your file:

1

2

javac TablePartA.java

java TablePartA

Please Ignore the Warnings in the Output.

Exercise B: List HBase Tables

This exercise lists all the tables created in the previous part. We have provided a boilerplate for this exercise in the following file: **TablePartB.java**.

We use the following commands to run your file:

1

2

javac TablePartB.java

java TablePartB

Your output should contain:

```
food
powers
```

Exercise C: Populate HBase Table with Data

This exercise will insert data in the "powers "table created in the previous part A with the following schema according to **input.csv**. You can assume that **input.csv** is in the same folder as the java files. We have provided a template for this exercise in the following file: **TablePartC.java**.

row key	personal		professional		custom
	hero	**power**	**name**	**xp**	**color**
row1					

We use the following commands to run your file:

1

2

javac TablePartC.java

java TablePartC

Exercise D: Read Data

In this exercise, you will read some of the data in the populated "powers" table created in part C. We have provided a template for this exercise in the following file: **TablePartD.java**.

Read the values of the given attributes in the following row ids, in this order:

Id: "row1", Values for (hero, power, name, xp, color)

Id: "row19", Values for (hero, color)

Id: "row1", Values for (hero, name, color)

The following is an example of the output lines. Please use spaces between the words. Your output should contain three lines for this exercise.

```
hero: no, color: green
```

We use the following commands to run your file:

1

2

javac TablePartD.java

java TablePartD

Exercise E: Scan Data

This exercise will scan the data in the populated "powers" table created in part C. We have provided a template for this exercise in the following file: **TablePartE.java**.

We use the following commands to run your file:

1

2

javac TablePartE.java

java TablePartE

The output should look something like the below, but you should print all the rows.

```
keyvalues={row1/custom:color/1520644330258/Put/vlen=5/seqid=0, row1/personal:hero/1520644330258/Put/vlen=3/seqid=0, row1/personal:power/152064433025
8/Put/vlen=3/seqid=0, row1/professional:name/1520644330258/Put/vlen=6/seqid=0, row1/professional:xp/1520644330258/Put/vlen=3/seqid=0}
keyvalues={row10/custom:color/1520644330298/Put/vlen=4/seqid=0, row10/personal:hero/1520644330298/Put/vlen=2/seqid=0, row10/personal:power/152064433
0298/Put/vlen=5/seqid=0, row10/professional:name/1520644330298/Put/vlen=7/seqid=0, row10/professional:xp/1520644330298/Put/vlen=2/seqid=0}
keyvalues={row11/custom:color/1520644330301/Put/vlen=5/seqid=0, row11/personal:hero/1520644330301/Put/vlen=2/seqid=0, row11/personal:power/152064433
0301/Put/vlen=3/seqid=0, row11/professional:name/1520644330301/Put/vlen=6/seqid=0, row11/professional:xp/1520644330301/Put/vlen=2/seqid=0}
```

Exercise F: Data Versioning

HBase allows you to store multiple versions of the same cell. Each version is identified by *the timestamp*. In this exercise, you will first update the "color" qualifier of the "custom" column family for an existing row with the row key "row7" to "purple". Then, retrieve values of all columns for the row with key "row7" including all versions of the "color" column for the updated row.

We have provided a template for this exercise in the following file: **TablePartF.java**. **Don't use any additional libraries**.

Hint: Each cell of the table is identified by the row key, column family, column qualifier, and version.

We use the following commands to run your file:

1

2

javac TablePartF.java

java TablePartF

The output should look similar to the following and contain **6** lines in total:

row: row7, family: custom, qualifier: color, value: purple, timestamp: 1682728060475 hero, power, name, xp one row each and 2 versions for color , both pink and purple.

Exercise G: A JOIN SQL query in HBase

We want to run a SQL query in the populated "powers" table created in part C in this exercise. But HBase, being a NoSQL system, doesn't support SQL. Therefore you should write the nested loop to achieve the same results as if HBase supported SQL. The SQL query is as follows:

SELECT p.name, p.power, p1.name, p1.power, p.color

FROM powers AS p

INNER JOIN powers AS p1

ON p.color = p1.color

AND p.name <> p1.name;

Scan the data in the table first before iterating over it. We have provided a template for this exercise in the following file: **TablePartG.java**. **Don't use any additional libraries**.

We use the following commands to run your file:

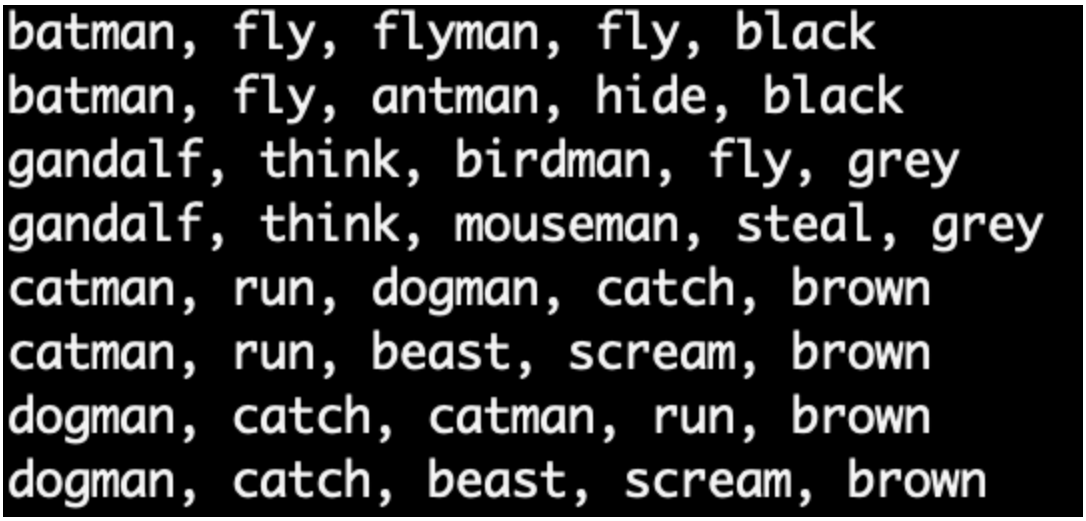
1

2

javac TablePartG.java

java TablePartG

The output should look similar to the following and contain **56** lines in total:



Exercise H: IMPLEMENTING SQL JOINS USING PHOENIX

Wouldn't it be easier if you could use SQL queries to run JOIN queries instead of looping through the records as in the previous exercise? Apache **Phoenix** allows you to do just that and more. Phoenix is an open-source **SQL** skin for HBase. It lets you run SQL queries to create tables, insert data, and query your HBase data instead of using the Hbase client API. In this exercise, you need to write SQL queries to perform the following functions: A) Map the Hbase **table** "powers" to a Phoenix **view** named "powers". This will then allow us to process the data through SQL. You may refer to the Phoenix FAQs to implement this section. B) Using the Phoenix view "powers", write a SQL JOIN query that returns the common power and the names of the heroes that share this power. Name the columns as "Name1, Name2, Power".

You need to write the SQL queries in the provided file named **phoenix.sql** and specify the exact database URL to run the queries in the provided file named **url.txt. Do not modify any other files while solving part H**.

!outputformat tsv

Do not delete this line. This sets the output format of the result set to TSV (Tab-Separated Values) to compare the query results obtained, with the autograder.

We use the following command to run your files:

1

/usr/local/phoenix/bin/sqlline.py "$(cat url.txt)" phoenix.sql

For instance given the records:

hero:yes , power:run, name:batman, xp:10 color:black hero:yes, power:run, name:antman, xp:10 color:pink hero:no, power:run, name:normalman, xp:10 color:brown hero;yes, power:swim, name:aquaman, xp:10 color:blue Your query should return the records with the exact column names :

1

2

3

4

5

6

7

8

```
+------------+------------+--------+
|  Name1   |  Name2   | Power |
+------------+------------+--------+
| batman    | batman    | run   |
| batman    | antman    | run   |
| antman    | antman    | run   |
| antman    | batman    | run   |
+------------+------------+--------+
```

Note that the third record is ignored in the output as "normalman" is not a hero.

Python submission

If you choose to do this assignment in Java, please skip this part and go to the **"Java submission" section above.

1. Requirements

This assignment will be graded based on **Python3.6 You may refer to the HappyBase documents for proper API information.**

2. Procedures

Step 1: Now we have to copy the template from our local folder to our container :

1

2

3

4

5

6

7

8

#open a new command prompt and get the container ID by running:

docker ps -a

#copy a local file into container

docker cp [OPTIONS] SRC_PATH CONTAINER:DEST_PATH

#docker cp C:\Users\lenovo\Desktop\MP7_HBase_Template\PythonTemplate 7cdd497f056
0:/PythonTemplate

#In the docker terminal change the current folder and run your files

cd PythonTemplate

Step 2: HappyBase relies on the Thrift server. Start Thrift using:

1

hbase thrift start &

Note that you may need to press "enter" after the following INFO message to continue:

INFO [main] server.Server: Started @1415ms"

Step 3: Finish the exercises by editing the provided templates files. Please note that you are **NOT allowed to import any additional libraries.**

- Each exercise has one or more code templates. All you must do is edit these files.

- The autograder (and you) will run the code on the provided Docker image.

Exercise A: Create HBase Tables

In this exercise, you will create two tables, as shown below. We have provided a template for this exercise in the following file: **TablePartA.py**.

Table 1: a table named “powers” in HBase with three column families as shown below. For simplicity, treat all data as **String** (i.e., don’t use any numerical data type).

row key	personal	professional	custom

Table 2: a table named “food” in HBase with two column families as shown below. For simplicity, treat all data as **String** (i.e., don’t use any numerical data type).

row key	nutrition	taste

We will use **python TablePartA.py** to run this part.

Your output should contain:

1

2

[thrift-worker-
0] client.HBaseAdmin: Operation: CREATE, Table Name: default:powers, procId: 9 completed

[thrift-worker-
0] client.HBaseAdmin: Operation: CREATE, Table Name: default:food, procId: 12 completed

Exercise B: List HBase Tables

This exercise will list all the tables created in the previous part. We have provided a template for this exercise in the following file: **TablePartB.py**. The command to run this part: **python TablePartB.py**.

Your output should contain:

1

[b'food', b'powers']

Exercise C: Populate HBase Table with Data

In this exercise, you will insert data in the “powers“ table created in the previous part A with the following schema according to **input.csv**. You can assume that **input.csv** is in the same folder as the python files. We have provided a template for this exercise in the following file: **TablePartC.py**.

row key	personal		professional		custom
	hero	power	name	xp	color
row1					

To run this part, we use the following command: **python TablePartC.py**.

Exercise D: Read Data

In this exercise, you will read some of the data in the populated “powers “ table created in part C. We have provided a template for this exercise in the following file: **TablePartD.py**.

Read the values of the given attributes in the following row ids, in this order:

Id: "row1", Values for (hero, power, name, xp, color)

Id: "row19", Values for (hero, color)

Id: "row1", Values for (hero, name, color)

We will use the following command to run this part: **python TablePartD.py**. The following is an example of output lines. Please use spaces between the words. Your output should contain three lines for this exercise.

1

2

3

hero: b'yes', power: b'fly', name: b'batman', xp: b'100', color: b'black'

hero: b'no', color: b'green'

hero: b'yes', name: b'batman', color: b'black'

Exercise E: Scan Data

This exercise will scan the data in the populated "powers "table created in part C. We have provided a template for this exercise in the following file: **TablePartE.py**. Enable timestamp during the scan to produce the accompanying timestamps in the output.

We use the following commands to run your file: **python TablePartE.py**. The output should look similar to the following, but you should print all rows:

1

2

3

4

5

6

7

Found: b'row1', {b'custom:color': (b'black', 1645021130731), b'personal:hero': (b'yes', 1645021130731), b'personal:power': (b'fly', 1645021130731), b'professional:name': (b'batman', 1645021130731), b'professional:xp': (b'100', 1645021130731)}

Found: b'row10', {b'custom:color': (b'grey', 1645021130801), b'personal:hero': (b'no', 1645021130801), b'personal:power': (b'think', 1645021130801), b'professional:name': (b'gandalf', 1645021130801), b'professional:xp': (b'86', 1645021130801)}

Found: b'row11', {b'custom:color': (b'brown', 1645021130806), b'personal:hero': (b'no', 1645021130806), b'personal:power': (b'run', 1645021130806), b'professional:name': (b'catman', 1645021130806), b'professional:xp': (b'84', 1645021130806)}

Found: b'row12', {b'custom:color': (b'brown', 1645021130812), b'personal:hero': (b'no', 1645021130812), b'personal:power': (b'catch', 1645021130812), b'professional:name': (b'dogman', 1645021130812), b'professional:xp': (b'47', 1645021130812)}

Found: b'row13', {b'custom:color': (b'white', 1645021130816), b'personal:hero': (b'maybe', 1645021130816), b'personal:power': (b'stink', 1645021130816), b'professional:name': (b'eggman', 1645021130816), b'professional:xp': (b'58', 1645021130816)}

.....

.....

Exercise F: Data Versioning

HBase allows you to store multiple versions of the same cell. Each version is identified by *the timestamp*. In this exercise, you will first update the "color" qualifier of the "custom" column family for an existing row with the row key "row7" to "purple". Then, retrieve values

of all columns for the row with key "row7" including all versions of the "color" column for the updated row.

We have provided a template for this exercise in the following file: **TablePartF.py**.

Hint: Each cell of the table is identified by the row key, column family, column qualifier, and version.

We use the following commands to run your file: **python TablePartF.py**

The output should look similar to the following and contain **6** lines in total:

row: b'row7', column family:qualifier: b'custom:color', value: b'purple', timestamp: 1682730962958

hero, power, name, xp one row each and 2 versions for color , both pink and purple.

Exercise G: A JOIN SQL query in HBase

We want to run an SQL query in the populated "powers" table created in part C in this exercise. But HBase, being a NoSQL system, doesn't support SQL. Therefore you should write the nested loop to achieve the same results as if HBase supported SQL. The SQL query is as follows:

1

2

3

4

5

SELECT p.name, p.power, p1.name, p1.power, p.color

FROM powers AS p

INNER JOIN powers AS p1

    ON p.color = p1.color

    AND p.name <> p1.name;

Scan the data in the table first before iterating over it. We have provided a template for this exercise in the following file: **TablePartG.py**. To run this part, we use **python TablePartG.py**. **Do not use any additional libraries**.

The output should look similar to the following and contains **56** lines in total:

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

b'batman', b'fly', b'flyman', b'fly', b'black'

b'batman', b'fly', b'antman', b'hide', b'black'

b'gandalf', b'think', b'birdman', b'fly', b'grey'

b'gandalf', b'think', b'mouseman', b'steal', b'grey'

b'catman', b'run', b'dogman', b'catch', b'brown'

b'catman', b'run', b'beast', b'scream', b'brown'

b'dogman', b'catch', b'catman', b'run', b'brown'

b'dogman', b'catch', b'beast', b'scream', b'brown'

b'eggman', b'stink', b'charming', b'smile', b'white'

b'eggman', b'stink', b'goatman', b'jump', b'white'

b'eggman', b'stink', b'elisa', b'freeze', b'white'

b'fishman', b'swim', b'brave', b'fight', b'blue'

b'birdman', b'fly', b'gandalf', b'think', b'grey'

b'birdman', b'fly', b'mouseman', b'steal', b'grey'

b'flyman', b'fly', b'batman', b'fly', b'black'

b'flyman', b'fly', b'antman', b'hide', b'black'

b'sofaman', b'stay', b'superman', b'run', b'red'

b'sofaman', b'stay', b'appleman', b'eat', b'red'

b'sofaman', b'stay', b'spiderman', b'fly', b'red'

b'beast', b'scream', b'catman', b'run', b'brown'

b'beast', b'scream', b'dogman', b'catch', b'brown'

b'goblin', b'hide', b'shrek', b'stink', b'green'

b'goblin', b'hide', b'spinachman', b'grow', b'green'

b'goblin', b'hide', b'robinhood', b'steal', b'green'

b'superman', b'run', b'sofaman', b'stay', b'red'

b'superman', b'run', b'appleman', b'eat', b'red'

b'superman', b'run', b'spiderman', b'fly', b'red'

.....

.....

Exercise H: IMPLEMENTING SQL JOINS USING PHOENIX

Wouldn't it be easier if you could use SQL queries to run JOIN queries instead of looping through the records as in the previous exercise? Apache **Phoenix** allows you to do just that and more. Phoenix is an open-source **SQL** skin for HBase. It lets you run SQL queries to create tables, insert data, and query your HBase data instead of using the Hbase client API. In this exercise, you need to write SQL queries to perform the following functions: A) Map the Hbase **table** "powers" to a Phoenix **view** named "**powers**". This will allow us to process the data through SQL. You may refer to the Phoenix FAQs to implement this section. B) Using the Phoenix view "powers", write a SQL JOIN query that returns the common power and the names of the heroes that share this power. Name the columns as "Name1, Name2, Power".

You need to write the SQL queries in the provided file named **phoenix.sql** and specify the exact database URL to run the queries in the provided file named **url.txt. Do not modify any other files while solving part H**.

!outputformat tsv **Do not delete this line**. This sets the output format of the result set to TSV (Tab-Separated Values) to compare the query results obtained, with the autograder.

We use the following command to run your files:

1

/usr/local/phoenix/bin/sqlline.py "$(cat url.txt)" phoenix.sql

For instance given the records: ***hero:yes , power:run, name:batman, xp:10 color:black hero:yes, power:run, name:antman, xp:10 color:pink hero:no, power:run, name:normalman, xp:10 color:brown hero;yes, power:swim, name:aquaman, xp:10 color:blue*** Your query should return the records with the correct column names :

1

2

3

4

5

6

7

8

```
+------------+------------+--------+
|  Name1   |  Name2   | Power |
+------------+------------+--------+
| batman    | batman    | run   |
| batman    | antman    | run   |
| antman    | antman    | run   |
| antman    | batman    | run   |
+------------+------------+--------+
```

Note that the third record is ignored in the output as "normalman" is not a hero.

==========

How to Submit

For this MP, compress all your files to a .zip file named "**MP7.zip**" and submit your "**MP7.zip**". Please do not upload the entire MP7 folder.

For the **Java** version, you should compress the seven java files, phoenix.sql, and url.txt.

For the **Python** version, you should compress the seven Python files, phoenix.sql and url.txt.

The autograder will run your code using the command lines in instructions and compare the files produced by your code files to the expected outputs.

If your code files fail to produce precisely the same results as the instructions, you may not get points for parts A, B, C, D, E, F, G, and H.

How to submit

When you're ready to submit, you can upload files for each part of the assignment on the "My submissions" tab.

Like

Dislike

Report an issue